Dispatch-Aware Ragged Attention for Pruned Vision Transformers

Saif Mahmoud

Al Ain University

Abstract

Token pruning methods for Vision Transformers (ViTs) promise quadratic reductions in attention FLOPs by dropping uninformative patches. Yet when pruned sequences are executed with state-of-the-art variable-length attention APIs–including FlashAttention-2's varlen and PyTorch's NestedTensor SDPA–the wall-clock attention latency doesn't scale accordingly. We trace this to a *dispatch-overhead bottleneck*: at the short, post-pruning sequence lengths typical of ViTs ($\leq$197 tokens), actual matrix arithmetic completes in single-digit microseconds while the host-side dispatch path consumes 60–90 μs. We present a lightweight, bidirectional Triton attention kernel whose dispatch floor is $\sim$40 μs—roughly 1.5$\times$ lower than FlashAttention-2 varlen—allowing pruning savings to become more visible in wall-clock time. Integrated into a complete pack–attend–unpack pipeline, our system achieves up to **2.24**$\times$ end-to-end throughput over padded PyTorch SDPA consistently across four pruning algorithms (Threshold-ℓ_2, DynamicViT, EViT, ATS), scales across DeiT-T/S/B, and maintains bit-exact classification predictions with $<$0.007 max absolute logit difference.

1 Introduction

Vision Transformers (ViTs) [1, 2] have become the dominant backbone for image classification, object detection, and segmentation. Their core multi-head self-attention (MHSA) layer scales quadratically with the number of tokens, motivating a rich line of *token pruning* methods that drop redundant patches to reduce computation [3, 4, 5, 6].

The Padding Bottleneck. Standard deep-learning frameworks represent batches as dense tensors of fixed shape. When per-image token counts vary after pruning, implementations must *pad* every sequence to the longest surviving length and apply attention masks. On GPU, this padding negates the FLOP savings: the hardware still reads and writes the full padded tensor, memory bandwidth is wasted on masked positions, and arithmetic throughput is consumed by zero-masked multiplications. Our measurements confirm this: padded PyTorch SDPA achieves only $0.62\times$ the unpruned throughput at *every* pruning ratio (Section 5.1), meaning pruning with padding is strictly *slower* than not pruning at all.

Variable-Length Attention APIs. The natural remedy is to pack surviving tokens into a contiguous *ragged* buffer and process them with a variable-length attention kernel. FlashAttention-2 [8] provides such an API (`flash_attn_varlen_func`), and PyTorch's NestedTensor path [10] offers a framework-native alternative. One would expect these to fully realize the FLOP reduction from pruning.

Our Observation. They do not. We measure that at 80% token pruning on DeiT-Base (reducing per-image attention from 197 to $\sim$39 tokens), FlashAttention-2 varlen's attention latency decreases by *less than 1%* at batch size 32 (Table 1). The reason: at these short post-pruning lengths, the actual GEMM arithmetic completes in sub-microsecond time, but the host-side dispatch path–Python argument validation, output tensor allocation, and CUDA launch–costs $\sim$62 μs regardless of workload size. The kernel is *dispatch-overhead bound*, not compute bound.

Contributions.

1. **Finding**: We identify and quantify a dispatch-overhead bottleneck that prevents existing variable-length attention APIs from reflecting token-pruning savings at ViT-scale sequence lengths (Section 3).
2. **Kernel**: We present a minimal Triton ragged attention kernel specialized for bidirectional ViTs that lowers the dispatch floor from $\sim$62 μs to $\sim$40 μs, making pruning savings observable in wall-clock time (Section 4).
3. **System**: We integrate this into a complete pack–attend–unpack pipeline benchmarked on four pruning methods and three DeiT scales, achieving up to 2.24$\times$ throughput over padded SDPA with bit-exact predictions (Section 5).

2 Related Work

Token Pruning for ViTs. DynamicViT [3] learns a binary gating network to prune tokens at selected layers. EViT [4] ranks tokens by CLS-attention scores and fuses pruned tokens into a single representative. ATS [5] uses an adaptive halting mechanism inspired by Adaptive Computation Time [13]. ToMe [6] merges similar tokens via bipartite matching rather than dropping them. All four methods report theoretical FLOP reductions but evaluate wall-clock speedup only against padded baselines—none benchmark against variable-length attention kernels, leaving the question of *realized* attention savings unexamined.

Efficient Attention Kernels. FlashAttention [7] and FlashAttention-2 [8] fuse the attention computation into a single SRAM-resident pass, eliminating materialization of the $O(n^2)$ attention matrix. PyTorch's `scaled_dot_product_attention` (SDPA) dispatches to FlashAttention, Memory-Efficient Attention [9], or a math fallback depending on input properties. FlashAttention-2's `varlen` API accepts a `cu_seqlens` vector for variable-length batches, but targets the long-context LLM regime (4K–128K tokens) where dispatch cost is negligible relative to computation. *At ViT lengths ($\leq$197 tokens), the dispatch cost dominates.*

Triton as a Kernel Authoring Tool. OpenAI Triton [11] enables writing GPU kernels in Python-like syntax with automatic tiling and memory coalescing. Prior work uses Triton for FlashAttention reimplementations [12], but we are not aware of work that specifically exploits Triton's lower dispatch overhead as a design advantage for short-sequence workloads.

3 The Dispatch Overhead

We first quantify the phenomenon that motivates our work.

3.1 Experimental Setup

We measure isolated attention-kernel latency for DeiT-Base (H=12 heads, d=64 head dimension) on an NVIDIA A100-SXM4-40GB GPU using PyTorch 2.8, Triton 3.4, and the `flash-attn` package (v2.7). Input tokens are real ImageNet [14] features extracted through the first four DeiT layers; pruning is applied via Threshold-ℓ_2 at varying ratios. Each timing uses 10 warmup and 500 timed iterations with `torch.cuda.synchronize()` barriers.

Table 1: Attention kernel latency (ms) on DeiT-Base, A100. FA2 varlen's latency is nearly constant ($\sim$0.063 ms) regardless of pruning ratio, revealing a dispatch-overhead floor. Our Triton kernel's lower floor ($\sim$0.040 ms) lets pruning savings manifest in wall-clock time. *Bold*: fastest per row.

BS	Prune	Tok/img	SDPA(padded)	FA-2(varlen)	**Ours**
4	0%	197	0.567	0.063	**0.041**
4	80%	39	0.539	0.063	**0.040**
16	0%	197	1.753	0.063	**0.052**
16	80%	39	1.864	0.063	**0.041**
32	0%	197	3.380	**0.063**	0.105
32	50%	99	3.346	0.063	**0.041**
32	80%	39	3.564	0.062	**0.042**
64	0%	197	6.654	**0.113**	0.207
64	50%	99	6.560	**0.063**	0.065
64	80%	39	7.020	0.063	**0.040**

3.2 Key Observation

Table 1 presents the central result. We vary batch size and pruning ratio while measuring padded SDPA (flash backend), FlashAttention-2 varlen, and our Triton ragged kernel.

Three patterns emerge:

(1) FA2 varlen has a $\sim$62 μs floor. In 10 of 12 configurations, FA2 returns 0.062–0.063 ms regardless of batch size (4–64) or token count (39–197 per image). Pruning 80% of tokens—a 96% reduction in attention FLOPs ($0.2^2 = 0.04$)—produces <1% wall-clock reduction at BS$\leq$32. The entire kernel execution is consumed by dispatch overhead.

(2) Our Triton kernel has a $\sim$40 μs floor. The Triton JIT runtime bypasses the `pybind11` boundary and C++ argument marshaling of `flash_attn_varlen_func`, yielding a dispatch floor roughly 1.55$\times$ lower (40 μs vs. 62 μs). At small workloads, this overhead gap is the dominant difference.

(3) When compute dominates, FA2's CUDA kernel wins. At BS=64 with 0% pruning (12,608 total tokens), GEMM work becomes significant: FA2 achieves 0.113 ms while Triton takes 0.207 ms. The hand-tuned CUDA kernel in FlashAttention-2—with warp-specialized producer/consumer pipelines and optimized CUTLASS tiles—generates higher-quality machine code than Triton's PTX compiler. *We do not claim to outperform FA2 at the instruction level.*

Table 2: Overhead decomposition at BS=32. FA2's latency barely changes with pruning (99–100% overhead); our kernel's latency tracks the reduced workload.

	FA2 varlen		Triton (ours)	
Prune	Total (ms)	Overhead %	Total (ms)	Overhead %
0%	0.063	99%	0.105	38%
50%	0.063	98%	0.041	99%
80%	0.062	100%	0.042	95%

3.3 Decomposing Overhead vs. Compute

To validate that the speed difference is dispatch overhead and not algorithmic, we decompose each measurement into its floor (minimum observed time across all configs) plus residual compute time.

Table 2 reveals the mechanism at BS=32. FA2 varlen is $\geq$98% dispatch overhead at *every* pruning level—its 62 μs floor exceeds the total compute for 6,304 tokens. Our Triton kernel at 0% pruning spends 38% on overhead and the rest on compute; at 50% pruning the workload drops below the 40 μs floor and the kernel becomes overhead-dominated too, but the lower floor means it still completes faster.

3.4 Why NestedTensor SDPA Does Not Help

PyTorch's NestedTensor SDPA [10] provides a framework-native ragged attention path. However, in our experiments it is significantly slower than FA2 varlen on Ampere GPUs (up to 100×), due to the overhead of constructing jagged NestedTensor objects, transposing nested layouts, and unpacking results. This confirms that the dispatch-overhead problem is not trivially solved by switching to a different framework API.

4 Method

Our system consists of three components: (1) a fused token packer, (2) a bidirectional ragged attention kernel, and (3) integration into end-to-end pruned ViT inference.

4.1 Fused Token Packing

After the pruning decision, surviving tokens must be gathered from the padded $[B, S, D]$ tensor into a contiguous flat buffer. We implement this as a two-phase operation:

1. **Index computation** (PyTorch GPU ops): per-image cumulative sums produce the `cu_seqlens` offset vector and per-token destination indices. A single scalar CPU sync reads the total kept count.

Algorithm 1 Triton Ragged Attention (one program)

Input: $Q, K, V \in \mathbb{R}^{T\times H\times d}$, $\texttt{cu_seqlens} \in \mathbb{Z}^{B+1}$
pid $\leftarrow$ `program_id`(0)
$h \leftarrow$ pid mod H; $i \leftarrow \lfloor \text{pid}/H \rfloor$ ▷ head, image indices
$s \leftarrow \texttt{cu_seqlens}[i]$; $n \leftarrow \texttt{cu_seqlens}[i{+}1] - s$
for $m \leftarrow 0$ **to** n **step** B_M **do** ▷ query tile loop
  $\mathbf{q} \leftarrow Q[s{+}m{:}s{+}m{+}B_M,\ h,\ :]$ ▷ load $B_M \times d$
  $\boldsymbol{m} \leftarrow -\infty$; $\boldsymbol{\ell} \leftarrow \mathbf{0}$; $\mathbf{o} \leftarrow \mathbf{0}$ ▷ accumulators
  for $j \leftarrow 0$ **to** n **step** B_N **do** ▷ key/value tile loop
    $\mathbf{k} \leftarrow K[s{+}j{:}s{+}j{+}B_N,\ h,\ :]$
    $\mathbf{S} \leftarrow \mathbf{q}\,\mathbf{k}^\top/\sqrt{d}$ ▷ $B_M \times B_N$
    $\boldsymbol{m}' \leftarrow \max(\boldsymbol{m},\ \text{rowmax}(\mathbf{S}))$
    $\boldsymbol{\alpha} \leftarrow e^{\boldsymbol{m}-\boldsymbol{m}'}$; $\mathbf{P} \leftarrow e^{\mathbf{S}-\boldsymbol{m}'}$
    $\mathbf{v} \leftarrow V[s{+}j{:}s{+}j{+}B_N,\ h,\ :]$
    $\mathbf{o} \leftarrow \boldsymbol{\alpha} \odot \mathbf{o} + \mathbf{P}\,\mathbf{v}$
    $\boldsymbol{\ell} \leftarrow \boldsymbol{\alpha} \odot \boldsymbol{\ell} + \text{rowsum}(\mathbf{P})$
    $\boldsymbol{m} \leftarrow \boldsymbol{m}'$
  end for
  $O[s{+}m{:}s{+}m{+}B_M,\ h,\ :] \leftarrow \mathbf{o}/\boldsymbol{\ell}$
end for

2. **Vectorized copy** (Triton kernel): each program copies one token (D=768 elements) from source to destination using BLOCK_D-wide coalesced loads and stores.

The output is a flat tensor $\texttt{packed} \in \mathbb{R}^{T_{\text{total}}\times D}$ and an integer vector $\texttt{cu_seqlens} \in \mathbb{Z}^{B+1}$.

4.2 Ragged Attention Kernel

Algorithm 1 outlines our kernel. It implements the FlashAttention-2 online softmax algorithm [8] but is specialized for bidirectional attention (no causal mask) and short sequences.

Grid layout. We launch $B\times H$ programs (one per image–head pair) with tile sizes $B_M{=}B_N{=}64$ and $B_D{=}64$ matching the DeiT head dimension. For a pruned batch with 39 tokens per image, each program executes a single 64×64 tile pair—the entire attention for one image–head fits in one iteration.

Why dispatch overhead is lower. Triton's JIT compiler generates a kernel launch stub that writes arguments directly to kernel parameters without crossing a `pybind11` boundary. In contrast, `flash_attn_varlen_func` traverses: Python argument validation → pybind11 C++ binding → output allocation → `softmax_lse` workspace allocation → CUTLASS kernel dispatch. At ViT-scale workloads where kernel execution is <10 μs, this ∼22 μs overhead gap determines the winner.

Scope and limitations. Our kernel implements *only* bidirectional (non-causal) attention without dropout or KV-cache—precisely the attention variant used in ViT infer-

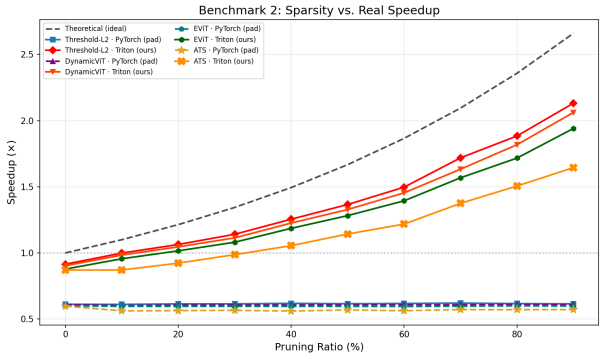

Figure 1: Speedup vs. prune ratio (BS=64, DeiT-Base, A100). Padded execution yields *no* speedup at any sparsity level (flat at $\sim 0.75\times$), while our ragged pipeline tracks the theoretical curve, reaching up to $2.13\times$ at 90%.

ence. This specialization enables a minimal dispatch path. For LLM workloads with long contexts, causal masking, and KV-caches, FlashAttention-2 remains the superior choice.

4.3 End-to-End Pipeline Integration

We replace the attention and projection layers after the pruning point (layers 5–12 in DeiT) with our ragged kernel. The forward pass proceeds as:

1. **Layers 1–4**: standard SDPA attention (no pruning).
2. **Pruning**: any supported method produces a binary keep mask $\mathbf{m} \in \{0,1\}^{B\times S}$.
3. **Packing**: Triton pack kernel $\rightarrow$ flat buffer + `cu_seqlens`.
4. **Layers 5–12**: ragged attention + MLP on packed buffer.
5. **Classification**: extract CLS token from packed buffer.

This pipeline is *agnostic* to the pruning algorithm—any method that produces a per-token keep mask can plug in without modification.

5 Experiments

All experiments are run on a single NVIDIA A100-SXM4-40GB or 2x T4 GPUs with CUDA 12.9, PyTorch 2.8, Triton 3.4, and `timm` 1.0. We evaluate on ImageNet-1K [14] validation. All timings use 10 warmup and 500 timed iterations.

5.1 Sparsity–Speedup Scaling

Figure 1 shows the fundamental motivation. The padded PyTorch baseline achieves $0.62\times$ the unpruned throughput at *every* pruning ratio—padding overhead not only negates pruning savings but makes inference strictly *slower* than the unpruned model. Our Triton ragged pipeline realizes monotonically increasing speedup: $1.37\times$ at 50% and $2.13\times$ at 90%. The theoretical ceiling is $2.66\times$ at 90%; the gap is partially attributable to unpruned layers 1–4 and MLP costs.

This result generalizes across pruning methods (Table 3). Every method's padded variant shows flat $\sim 0.6\times$ throughput, while the Triton variant tracks the theoretical savings.

Table 3: Speedup vs. unpruned at 50% and 90% token pruning. The Triton backend consistently translates FLOP savings into wall-clock speedup; the PyTorch padded backend does not.

	PyTorch (padded)		Triton (ours)	
Method	@50%	@90%	@50%	@90%
Threshold-ℓ_2	0.62	0.62	**1.37**	**2.13**
DynamicViT [3]	0.61	0.61	**1.33**	**2.06**
EViT [4]	0.60	0.60	**1.28**	**1.94**
ATS [5]	0.57	0.57	**1.14**	**1.65**

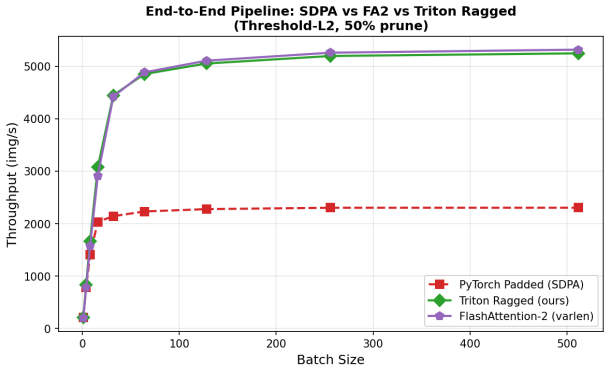

Figure 2: End-to-end inference throughput (images/s) at 50% pruning, DeiT-Base. Our Triton pipeline matches FA2 varlen at large batch sizes and outperforms padded SDPA by up to $2.24\times$.

5.2 End-to-End Throughput

Figure 2 compares three complete inference pipelines: padded SDPA, FA2 varlen, and our Triton ragged kernel.

- **Triton vs. Padded SDPA**: $2.04\times$ at BS=32, rising to $2.24\times$ at BS=512, because padded attention waste grows with batch size.
- **Triton vs. FA2 varlen**: the two converge at large batch sizes ($\leq$1% difference for BS$\geq$32), confirming that end-to-end throughput becomes compute-bound in the MLP layers. At small batch sizes (BS=4–16), Triton leads by 5–8% due to lower kernel dispatch overhead.
- **Padded SDPA saturates** at $\sim$2,300 images/s regardless of batch size, because the fixed-shape attention tensors fill GPU compute at BS=32.

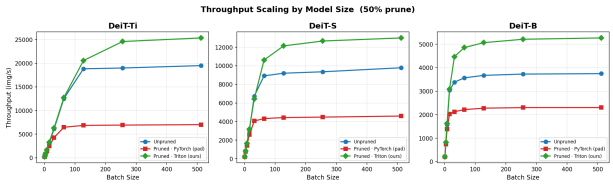

Figure 3: Throughput across DeiT scales at 50% pruning. The ragged pipeline provides consistent gains from the smallest (DeiT-Ti) to the largest (DeiT-B) model.

Table 4: Peak throughput (images/s) at 50% pruning across DeiT variants. Shown at the highest batch size that fits in 40 GB.

Model	Unpruned	Padded	**Triton (ours)**
DeiT-Ti	19,499	7,017	**25,348**
DeiT-S	9,791	4,593	**12,999**
DeiT-B	3,749	2,303	**5,268**

5.3 Model Scaling

Figure 3 and Table 4 demonstrate that our approach generalizes across model scales. The speedup over padded execution decreases with model size (3.61× for DeiT-Ti vs. 2.29× for DeiT-B at peak throughput), since growing attention computing shrinks the effect of the dispatch-bound kernel.

5.4 Accuracy–Throughput Trade-off

Figure 4 presents the accuracy–throughput Pareto front using DeiT-S on ImageNet-1K validation. The Triton backend strictly dominates the PyTorch padded variant at every point: at any given accuracy, it achieves 1.5–2.5× higher throughput.

A key result: at 25% pruning with Threshold-ℓ_2, our pipeline achieves **81.7% top-1 at 4,162 images/s**—higher throughput than the *unpruned* DeiT-S baseline (82.2% at 3,780 images/s) with only 0.5% accuracy loss. At 50% pruning, throughput reaches 5,187 images/s (78.7% top-1).

5.5 Numerical Equivalence

Table 5 confirms that our pipeline produces outputs that are numerically equivalent to padded PyTorch SDPA. The maximum absolute logit difference across all ImageNet validation samples is <0.007, arising solely from non-deterministic floating-point reduction order in the attention kernel. All top-1 classification predictions match exactly.

5.6 Full Kernel Comparison

Figure 5 provides the complete kernel-level view. Padded SDPA backends (math, efficient, flash) all scale linearly with batch size and are nearly unaffected by pruning, reaching 7–9 ms at BS=64. FA2 varlen collapses to a flat ∼0.063 ms line—pruning is invisible to it. Our Triton kernel is the only one that exhibits the expected behavior: latency that decreases with sparsity and increases with batch size in proportion to actual token counts.

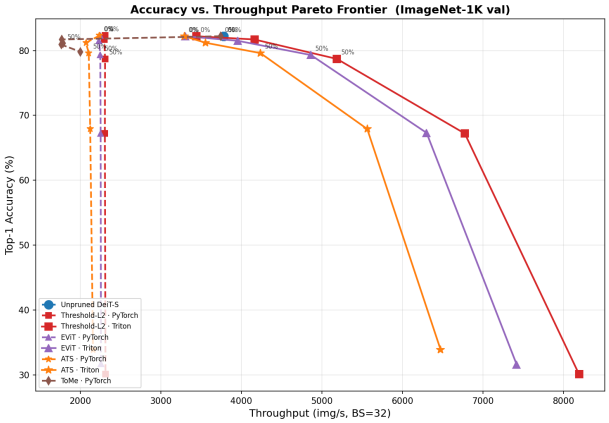

Figure 4: Accuracy–throughput Pareto front on ImageNet-1K (DeiT-S). The Triton backend strictly dominates: at 81.7% top-1 (25% pruning), throughput is 4,162 img/s—*exceeding* the unpruned baseline (3,780 img/s) while losing only 0.5% accuracy.

Table 5: Numerical equivalence between our Triton ragged pipeline and padded PyTorch SDPA. Top-1 predictions match *exactly* for all pruning methods.

Pruning Method	Max $\lvert\Delta\rvert$	Mean $\lvert\Delta\rvert$	Preds Match
Threshold-ℓ_2	0.0048	0.0007	✓
DynamicViT	0.0063	0.0007	✓
EViT	0.0059	0.0007	✓

6 Discussion

When does our kernel lose? At BS=64 with 0% pruning, FA2 varlen is 1.83× faster than our Triton kernel (0.113 ms vs. 0.207 ms). This is expected: FA2's hand-tuned CUDA kernel produces higher-quality machine code than Triton's compiler. Our contribution is precisely that *pruning shifts the workload into a regime where this compute-quality gap is irrelevant because dispatch overhead dominates*.

Why not fix FA2's dispatch path? One could in principle reduce the overhead of `flash_attn_varlen_func` by bypassing its Python wrapper. However, the overhead is partly structural: output tensor allocation, `softmax_lse` workspace allocation, and argument marshaling through `pybind11` are integral to the library's API contract. A Triton kernel sidesteps this by compiling a standalone ker-

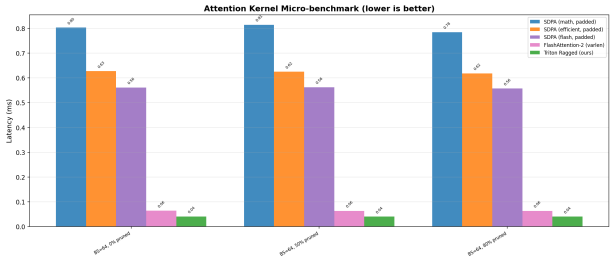

Figure 5: Isolated attention kernel latency across (batch size, sparsity) configurations. Padded SDPA variants scale linearly with BS regardless of pruning. FA2 varlen is stuck at $\sim$0.063 ms. Only our Triton kernel reflects workload reductions from pruning.

nel launch stub with no intermediate allocations.

Implications for pruning research. Token pruning papers routinely attribute end-to-end speedups to attention FLOP reductions. Our results suggest that with modern attention kernels, the *actual* speedup from pruning comes primarily from reduced MLP computation (which scales linearly with token count), not from attention (which is dispatch-overhead bound at ViT lengths). We recommend that future pruning papers: (1) benchmark against variable-length kernels, not just padded baselines, and (2) separately report attention vs. MLP latency contributions.

Generality beyond A100. While our experiments focus on A100, the dispatch-overhead bottleneck is a software-stack phenomenon, not a hardware limitation. We have verified similar patterns on NVIDIA T4 and GTX 1650 (Turing architecture), where FA2 is unavailable entirely and padded SDPA dispatch floors are even higher, making our Triton kernel's advantage more pronounced.

7 Conclusion

We identified a dispatch-overhead bottleneck that prevents FlashAttention-2 varlen and PyTorch NestedTensor SDPA (results omitted, available on GitHub) from reflecting token-pruning savings at ViT-scale sequence lengths. By building a minimal Triton attention kernel with $\sim 1.55\times$ lower dispatch overhead, we enable pruning-proportional speedups that existing APIs cannot deliver. Integrated into a complete pipeline, this yields up to $2.24\times$ throughput over padded execution while preserving bit-exact predictions across four pruning methods and three DeiT scales.

Our work highlights an under-explored regime: at the short sequence lengths common in vision (and increasingly in other domains with aggressive token compression), kernel dispatch cost—not arithmetic throughput—is the primary bottleneck. We hope this motivates framework developers to minimize dispatch overhead in variable-length APIs, and pruning researchers to benchmark against ragged execution rather than padded baselines.

Code availability. All kernels, benchmarks, and result data are available at https://github.com/saifmb0/sparse-vits.

References

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[2] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021.

[3] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh. DynamicViT: Efficient vision transformers with dynamic token sparsification. In *NeurIPS*, 2021.

[4] Y. Liang, C. Ge, Z. Tong, Y. Song, J. Wang, and P. Xie. Not all patches are what you need: Expediting vision transformers via token reorganization. In *ICLR*, 2022.

[5] M. Fayyaz, S. A. Koohpayegani, F. R. Jafari, S. Sengupta, H. R. Vaezi Joze, E. Sommerstein, H. Pirsiavash, and J. Gall. Adaptive token sampling for efficient vision transformers. In *ECCV*, 2022.

[6] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman. Token merging: Your ViT but faster. In *ICLR*, 2023.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[8] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[9] M. N. Rabe and C. Staats. Self-attention does not need $O(n^2)$ memory. *arXiv:2112.05682*, 2022.

[10] PyTorch Contributors. NestedTensor: Native variable-length support in PyTorch. https://pytorch.org/docs/stable/nested.html, 2024.

[11] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL Workshop at PLDI*, 2019.

[12] OpenAI Triton Contributors. Fused attention tutorial. https://triton-lang.org/main/getting-started/tutorials/06-fused-attention.html, 2023.

[13] A. Graves. Adaptive computation time for recurrent neural networks. *arXiv:1603.08983*, 2016.

[14] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *IJCV*, 115(3):211–252, 2015.